Ex:6

23/9/26

# Bidirectional RNN VS Feedforward RNN FOR Time - Series Prediction

## AIM:

To Implement a Bidirectional RNN (BIRNN) and a Feedforward Neural Network (FFNN) for time - Series Prediction using the Airline Passenger dataset and compare their performance using Mean Squared Error (MSE).

## ALGORITHM:

* Import the required python libraries.
* Load the Airline passenger dataset and normalize the data using MinMaxScaler.
* Create input sequences of 10 time steps and corresponding target values.
* Split the data into training and testing sets.
* Convert the data into pytorch tensors and create DataLoaders.
* Define the BiRNN and Feedforward Neural Network models.
* Train both models using MSE loss and Adam optimizer.

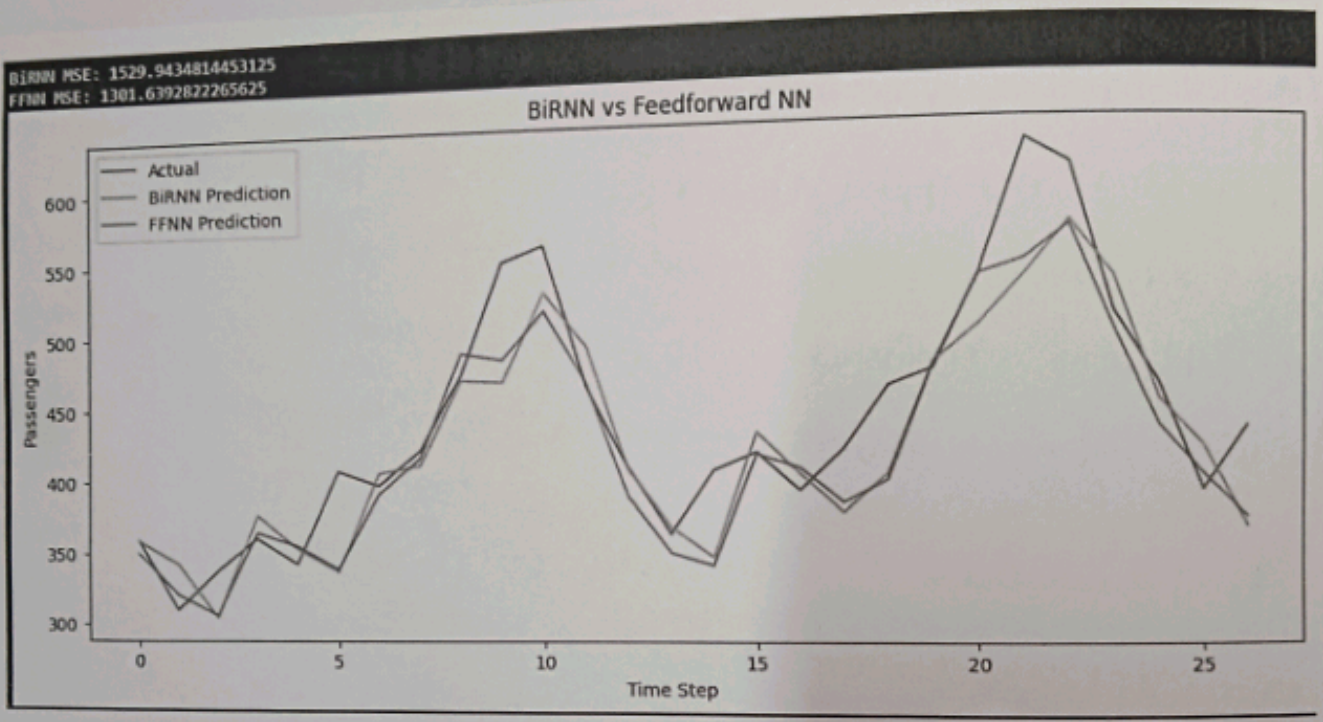
* Convert the predictions back to the original scale.

* Calculate MSE and plot the actual and predicted values for comparision.

CODE:

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt.
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/monthly-airline.csv")
data = pd.read_csv(url).iloc[:,1].values.astype("float32")
scaler = MinMaxScaler()
data = scaler.fit_transform(data.reshape(-1,1))
x = np.array([data[i:i+10] for i in range(len(data)-10)])
y = np.array([data[i+10] for i in range(len(data)-10)])
n = int(len(x)*0.8)
xtr, xts = torch.tensor(x[:n]), torch.tensor(x[n:])
ytr, yts = torch.tensor(y[:n]), torch.tensor(y[n:])
class BiRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(1,32, batch_first=True,
                          bidirectional=True)
        self.fc = nn.Linear(64,1)
```

BiRNN MSE: 1529.9434814453125
FFNN MSE: 1301.6392822265625



```
def train(model):
    for _ in range(50):
        opt.zero_grad()
        loss.backward()
        opt.step()
rnn, fnn = BiRNN(), FFNN()
train(rnn)
train(ffnn)
actual = scaler.inverse_transform(yte.numpy())
P1 = scaler.inverse_transform(P1)
P2 = scaler.inverse_transform(P2)
print("BiRNN MSE:", mean_squared_error(actual, P1))
print("FFNN MSE:", mean_squared_error(actual, P2))
plt.plot(actual, label="Actual")
plt.plot(P1, label="BiRNN")
plt.plot(P2, label="FFNN")
plt.legend()
plt.show()
```

RESULT:

BiRNN and FFNN were successfully implemented and compared. The predictions were visualized and their performance was evaluated using MSE.